



دانشگاه شهید باهنر کرمان





دانشگاه شهید باهنر کرمان

دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی

بخش مهندسی کامپیوتر

این پایان نامه با عنوان تشخیص قدرتمند بدافزارهای اندروید با استفاده از شبکه‌های عصبی ترنسفورمر توسط آقای علیرضا ایرانمنش دانشجوی رشته مهندسی کامپیوتر گرایش هوش مصنوعی و رباتیک با شماره دانشجویی ۴۰۱۱۵۵۰۱۵ تدوین شده است و در تاریخ ۱۴۰۴/۰۳/۱۹ با درجه خوب و نمره ۱۷.۵ مورد پذیرش هیئت محترم داوران قرار گرفت.

این پایان نامه هیچگونه مدرکی به عنوان فراغت از تحصیل دوره کارشناسی ارشد شناخته نمی‌شود.

سمت	نام و نام خانوادگی	امضاء
استاد راهنما	دکتر حمید میروزی	
داور اول	دکتر محمد جواد رستمی	
داور دوم	دکتر وحید ستاری	

معاون آموزشی و پژوهشی دانشکده:

دکتر سعید شجاعی

امضاء

نماینده تحصیلات تکمیلی:

دکتر وحید جمشیدی

امضاء

حق چاپ محفوظ و مخصوص به دانشگاه شهید باهنر کرمان است.

به نام خدا

مشور اخلاق پژوهش

با استعانت از خدای سبحان و اعتقاد راسخ به این که عالم محضر خداست و او بواره ناغیر بر اعمال ماست و به منظور انجام شایسته‌ی پژوهش‌های اصلی، تولید دانش جدید و بهسازی زندگی‌های بشر، ما دانشجویان و اعضای هیأت علمی دانشگاه پژوهشگاه‌های کشور:

- ☐ تمام تلاش خود را برای کشف حقیقت و حفظ حقیقت به کار خواهیم بست و از هر گونه جعل و تحریف و فعالیت‌های علمی پرهیزی کنیم.
- ☐ حقوق پژوهشگران، پژوهیدگان (انسان، حیوان، نبات و اشیا)، سازمان‌های صاحبان حق را بر رعایت می‌شناسیم و در حفظ آن‌ها می‌کوشیم.
- ☐ به مالکیت‌های و ممنوعی آثار پژوهشی ارجح می‌نیم، برای انجام پژوهشی اصیل اهتمام و در زنده‌د و از سرقت علمی و ارجاع نامناسب اجتناب می‌کنیم.
- ☐ ضمن پابندی به انصاف و اجتناب از هر گونه تبعیض و تعصب در کیه فعالیت‌های پژوهشی، رهیافتی اتخاذ خواهیم کرد.
- ☐ ضمن امانت‌داری، از منابع و امکانات اقتصادی، انسانی و فنی موجود، استفاده بهر و دراز خواهیم کرد.
- ☐ از انتشار غیر اخلاقی نتایج پژوهش، نظیر انتشار موازی، بهوشان و چندگانه‌گذاری پرهیزی کنیم.
- ☐ اصل همراه بودن و رازداری را محور تمام فعالیت‌های پژوهشی خود قرار می‌دهیم.
- ☐ در به فعالیت‌های پژوهشی به منافع ملی توجه کرده و برای تحقق آن‌ها می‌کوشیم.
- ☐ خویش را ملزم به رعایت کلیه بندهای علمی رسته خود، قوانین و مقررات، سیاست‌های حرفه‌ای، سازمانی، دولتی و راهبردی ملی و بهر مراحل پژوهش می‌دانیم.
- ☐ رعایت اصول اخلاق در پژوهش را اقدامی فرهنگی می‌دانیم و به منظور بالندگی این فرهنگ، به ترویج و اشاعه آن در جامعه اهتمام می‌ورزیم.



دانشگاه شاهرود

(affiliation)

(RNN و LSTM)

Area Under the ROC Curve

(Code Analysis)

Hybrid Approaches

Qualys (VMDR Mobile)

B

C

92.8%CNN90.6%SVMAUC = 0.9932 ± 0.003 F1 = 0.9823 ± 0.002 97.24% $\pm 0.5\%$ MAGNET

91.5%LSTM

API

AndroZooDrebin

VirusShareCICMalDroid

-
- ¹Android
 - ²Malware
 - ³Permissions
 - ⁴Machine Learning
 - ⁵Deep Learning
 - ⁶Model Interpretability
 - ⁷Generalization
 - ⁸Zero-Day
 - ⁹MAGNET(Multi-Modal Analysis for Graph and Network Threat Detection)
 - ¹⁰Transformer
 - ¹¹Attention Mechanism
 - ¹Dynamic Analysis
 - ²Static Analysis
 - ³API
 - ⁴Trojan
 - ⁵Spyware
 - ⁶Ransomware
 - ¹Zero-Day

MAGNET

2

MAGNET3

MAGNET

4

MAGNET

5

Statista

AUCF1 ScoreMAGNETOptunaPIRATES

MAGNET

n-gram

-
- ¹Trojan
 - ²Spyware
 - ³Ransomware
 - ⁴Zero-Day
 - ⁵Transformer
 - ⁶Optuna
 - ⁷Cross-Validation
 - ⁸Accuracy
 - ⁹Area Under Curve (AUC)
 - ¹⁰Static Analysis
 - ¹¹Dynamic Analysis
 - ¹Convolutional Neural Networks (CNNs)
 - ²Recursive Neural Networks (RNNs)
 - ³Long Short-Term Memory (LSTM)
 - ¹Ransomware
 - ²Cryptor
 - ³Locker

(Backdoor)

DoS

FTPFTP

⁴Mac OS X

⁵Trojan Horse

¹Spyware

²عبور رمزهای یا بانکی اطلاعات مانند

³Adware

⁴میکنند در آشنایی تبلیغ نمایش طریق از رایگان نرم افزارهای برخی

⁵Permission System

⁶Task

⁷Protection Level

API

(Normal)

(Dangerous)

.debAPPX.dexAPK APK

APKAPK.apkAPK

APK

++C

APK.dexART

APIMAGNET

⁸Marshmallow

⁹Runtime Permissions

¹AndroidManifest.xml

²پیامک‌ها مکانی، موقعیت مخاطبین، مانند

³میکروفون یا دوربین به دسترسی مانند

⁴Android Package Kit

⁵Android Package Kit

⁶AndroidManifest.xml

⁷اوبونتو مانند

⁸Google Play

⁹Sideloaded

¹Source Code

²Dalvik

³(Android Runtime)

⁴بایت‌کد یا باینری کدهای کردن دیس اسمبل مانند

⁵Multi-Modal Data

API SDK APK

Datatorch_geometricPyTorch Geometric

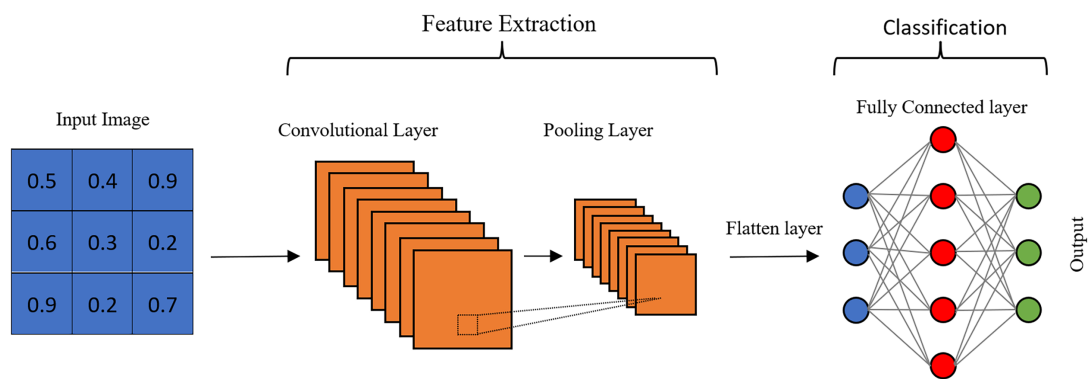
LSTMRNNAPIPyTorchLongTensorCPUAPI

load_sequence_data
load_graph_data
load_processed_data
FusionMAGNET

API

ReLU

-
- ¹Activities
 - ²Services
 - ³Broadcast Receivers
 - ⁴Content Providers
 - ⁵AndroidManifest.xml
 - ⁶PyTorch
 - ⁷torch.Tensor
 - ⁸Call Graph
 - ⁹Control Flow Graph, CFG
 - ¹⁰Data Dependency Graph
 - ¹¹Intents
 - ¹²Nodes
 - ¹Padding
 - ¹Convolutional Neural Networks, CNNs
 - ²Pooling
 - ³Fully Connected
 - ⁴CNN
 - ⁵End-to-End



(RNN 3 LSTM)

APIRNNRNN

LSTMGRULSTMLSTM RNN

LSTMAPIGRULSTM

GNN

GNNGNN

(Aggregation)

(Update)

GNN

GNNDataAPIGNN

¹ Recurrent Neural Networks, RNNs

² Feedforward

³ Hidden State

⁴ Vanishing/Exploding Gradient

⁵ Long Short-Term Memory, LSTM

⁶ Gated Recurrent Unit, GRU

⁷ Memory Cell

⁸ Graph Neural Networks, GNNs

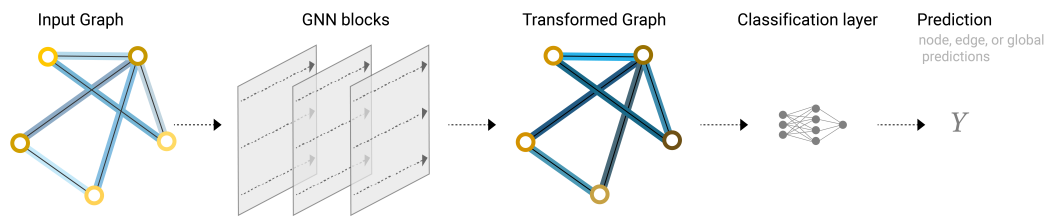
⁹ Nodes

¹⁰ Edges

¹ Graph Convolutional Network (GCN)

² Graph Attention Network (GAT)

³ torch_geometric



RNN/LSTMAttention Is All You Need

API

GPTBERT

MAGNET

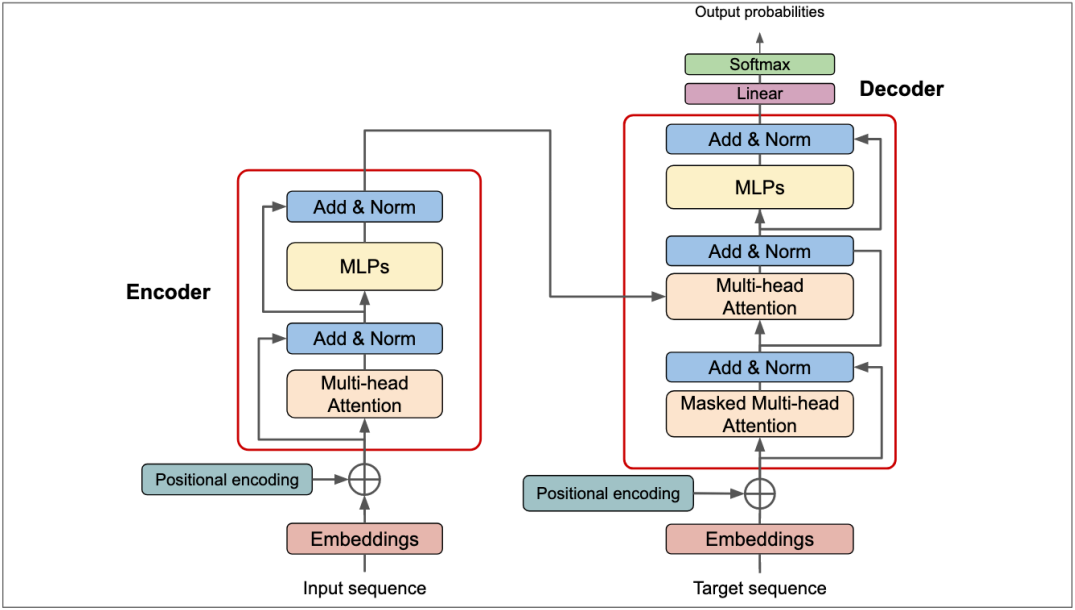
APIGraph Transformers

MAGNET

• .9823F1 Score97.24MAGNET

SVM

-
- ¹ Attention Mechanism
 - ² Self-Attention
 - ³ Multi-Head Attention
 - ⁴ Encoder
 - ⁵ Decoder
 - ⁶ Feed-Forward Networks
 - ⁷ Residual Connections
 - ⁸ Layer Normalization
 - ⁹ Large Language Models, LLMs
 - ¹⁰ Fine-tuning
 - ¹¹ Multi-Modal Analysis for Graph and Network Threat Detection
 - ¹² Embeddings
 - ¹ Fusion
 - ² Cross-Attention
 - ³ Support Vector Machine, SVM
 - ⁴ Classification
 - ⁵ Regression
 - ⁶ Hyperplane



Support VectorsSVM

SVM

SVMSVMSVM

SVMAPI SVM

$O(n^3)O(n^2)$

BaselineSVMCNN

⁷Margin
¹Linear SVM
²Non-linear SVM
³Kernel Trick
⁴Polynomial
⁵Radial Basis Function, RBF
⁶Sigmoid
¹Overfitting

MAGNET

(k-fold Cross-Validation)

k

k

Validation Setik-1ki

kF1 Score

scikit-learnStratifiedKFoldKFoldk = 5

MAGNET

²Cross-Validation
³Overfitting
¹5-fold Cross-Validation
²Overfitting
³Underfitting
⁴Underfitting
⁵num_layers
⁶embedding_dim
⁷Overfitting
⁸Regularization
¹Dropout

weight_decay=0.01

0.001 Adam RMSProp AdaGrad Adam Learning Rate Adam Stochastic Gradient Descent, SGD
GNN Adam MAGNET train_and_evaluate_magnet 0.01 weight_decay L2

PIRATES

(Ships)

(Leader)

(Top Ships)

PIRATES

²Early Stopping
³Weight Decay
⁴Loss Function
⁵Hyperparameters
⁶Adam
⁷Adaptive Moment Estimation
¹Pirate-Inspired Robust Adaptive Trajectory Exploration Strategy

OptunaPruningOptuna OptunaFrameworkOptuna
OptunaOptunaDropoutOptuna

MAGNET

Accuracy

$$= \frac{+}{+ + +}$$

TP (True Positive)

TN (True Negative)

FP (False Positive)

FN (False Negative)

MAGNET

RecallPrecisionF1 ScoreF1

(Precision)

$$= \frac{\quad}{+}$$

(TPR)

$$= \frac{\quad}{+}$$

¹Random Search
²Grid Search
³Tree-structured Parzen Estimator (TPE)
⁴مانند Median Pruning یا Successive Halving
⁵embedding_dim

$$= 2 \times \frac{\times}{+} = \frac{2 \times}{2 \times + +}$$

FPFN · .9823 F1 Score Recall FNPrecision FPF1

Area Under the ROC Curve

Recall

AUC

AUC = 1

AUC = 0.5

AUC < 0.5

MAGNET 0.91AUC(0.9123)F1 ScoreAUC

MAGNETAUCF1 Score

AndroidManifest.xml

SVMDrebin(Permissions)

¹Receiver Operating Characteristic, ROC²True Positive Rate, TPR³False Positive Rate, FPR⁴Area Under the Curve, AUC¹Static Analysis

Intent(Components)

(Code Analysis)

NativeDalvik/ART

APIAPIAPIAPI

Taint AnalysisDFGCFG(Data/Control Flow Analysis)

OpcodesOpcode

GNN

JNIC/C++NativeNative

SandboxDynamic Analysis

TaintTaintTaintDroid

²مانند Activities, Services, Receivers, Providers)

¹Call Graph

²Obfuscation

³رشته‌ها رمزنگاری زائد، کدهای درج توابع، و متغیرها نام تغییر مانند

⁴Dynamic Code Loading

¹(System Call Tracing)

²Dynamic Taint Analysis

IPC&C

DoSCPU

Joe Sandbox Mobile Mobile Sandbox AndroPyTool DroidBox

Hybrid Approaches

MAGNET

WIDOMW&SDenning

TaintDroid SVM Drebin

GNN API RNN/LSTM CNN

APICNN

MAGNET

¹Limited Code Coverage

²Environment Detection

³Flushot Plus

⁴ID intrusion detection system

SVMGNNRNN/LSTMCNNAPK

AUCF1 ScoreOptunaAdamDropout

MAGNET

(SAST, DAST, MTD)

(NowSecure Platform)

Qualys (VMDR Mobile)

VMDR Mobile

CI/CD

(NowSecure و Qualys)

NowSecure

NowSecure

MDM/EMMQualys

SASTCheckmarx

¹SAST+DAST+behavior

²inventory+vuln+response

³SAST

PyTorch 1.9.0

PyTorch Geometric 1.7.0

Scikit-learn 0.24.2

Optuna 2.10.0

CUDA VRAM GPU: NVIDIA RTX 3090

CPU: Intel Xeon E5-2690 v4

DDR4 RAM

$$\mathbf{x} \in \mathbb{R}^{128}$$

$$x = \frac{x - \mu}{\sigma},$$

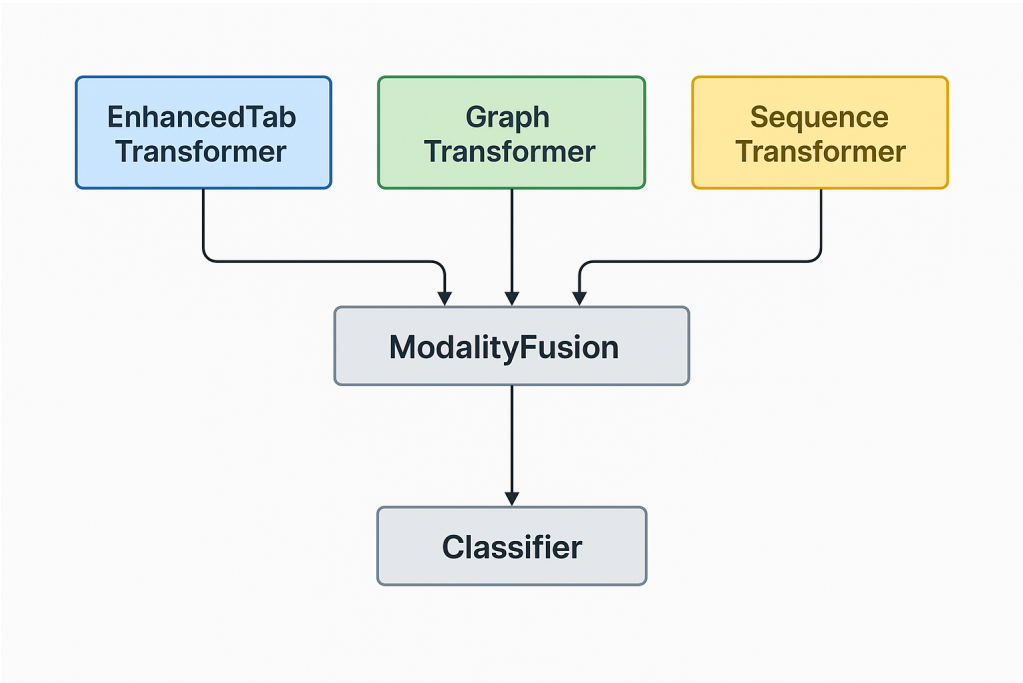
$$\sigma \mu$$

¹ میانگین = ۱۲.۳، معیار انحراف = ۴.۱
مگابایت ² = ۱.۹ معیار انحراف مگابایت، میانگین = ۲.۸
¹ $\mu = ۱۲.۳$ مجوزها: برای مثلاً $\sigma = ۴.۱$

$$\mathbf{e}_{uv} \in \mathbb{R}^{32} E \mathbf{x}_v \in \mathbb{R}^{64} VG = (V, E)$$

Data

$$\mathbf{x} \in \mathbb{Z}^{100}$$



x_i

$$\mathbf{e}_i = ((Wx_i + b)),$$

$$b \in \mathbb{R}^{64} W \in \mathbb{R}^{1 \times 64}$$

$$0.025 = \text{اسیاری سیتی میانگین}^2$$

۳ = پد توکن

$$\mathbf{p} \in \mathbb{R}^{1 \times 128 \times 64}$$

$$xbatchsize \times inputdim$$

TransformerConv

$$\mathbf{h}_v = W\mathbf{x}_v + b,$$

$$W \in \mathbb{R}^{64 \times 64}$$

$$data.xedgeindexededgeattr$$

$$PE(pos,2i)=\sin\left(\frac{pos}{10000^{2i/d}}\right),PE(pos,2i+1)=\cos\left(\frac{pos}{10000^{2i/d}}\right),$$

$$d=64pos$$

$$\overline{seq}$$

$$(Q,K,V)=\left(\gamma\cdot\frac{QK^T}{\sqrt{d_k}}\right)V,$$

$$d_k = \text{ / } = Q,K,V\gamma$$

$$\overline{x}$$

$$x$$

$$\gamma$$

$$\mathbf{z} = ([\mathbf{z}, \mathbf{z}, \mathbf{z}]),$$

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)],$$

$$\beta_2 = . \beta_1 = . \eta = .$$

num_heads

num_layers

dropout

$$. \quad = \text{dropout} \quad = \text{num_layers} \quad = \text{num_heads}$$

$$\frac{+}{+++}$$

$$2 \cdot \frac{\dot{+}}{+}$$

LSTM:CNN: 92.8%SVM: 90.6%AUC = 0.981 ± 0.003 F1 = 0.9823 ± 0.0029 7.24% $\pm 0.5\%$

91.5%

(Real-time Detection)

(Adversarial Robustness)

SVMDrebin

GNNAPIRNN/LSTMCNN

APICNN

99.32%98.23%F1 Score97.24%

11,12403271,4514,6416,092

32

4

1

128

0.2029

16

0.00215

0.00107

3

64

4

1

128

0.2

16

0.0019

0.0011

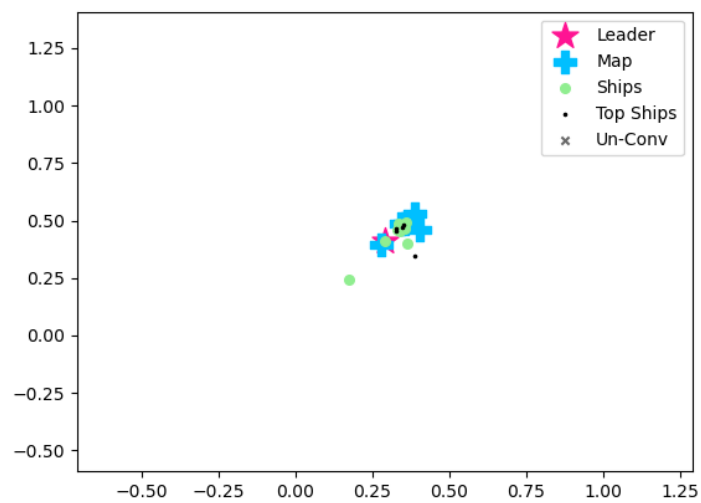
10

BSF

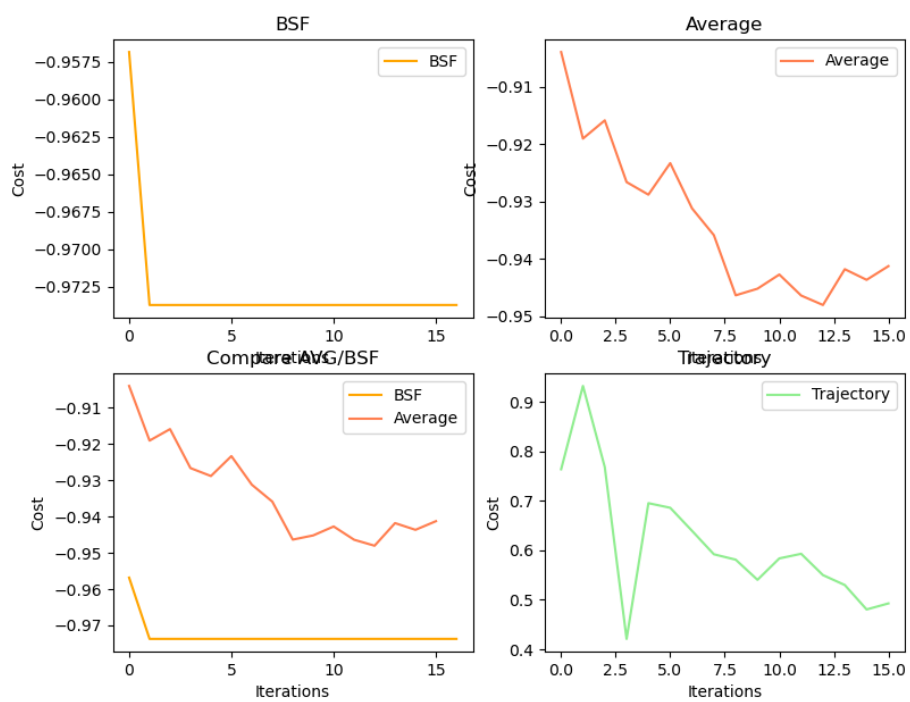
Average

Compare AVG/BSF

Trajectory

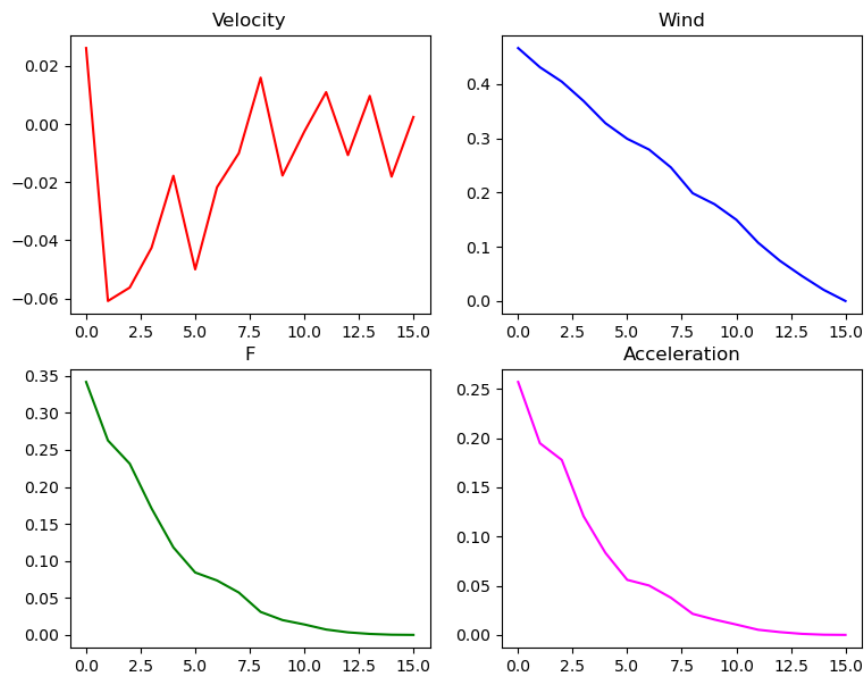


BSF/AVG/Trajectory



Velocity

Wind/Velocity



Wind

F

Acceleration

۲۱۹.۷۶

۲.۱۸

۰.۰۰۰۱۹۴

۰.۳۲۸۶

0.00194

0.3286

3.795e-05

F1 Score

Optuna

RBFSVM

100

¹Random Forest

6100XGBoost

ANN

89.2%

95.8%

3.8.5

1.9.0

1.7.0

0.24.2

1.3.31.21.2

243090

32426905

128

F1 Score

$$= \frac{+}{+ + +}$$

True Positive

True Negative

False Positive

False Negative

$$= \frac{\cdot}{+}$$

$$= \frac{\cdot}{+}$$

F1 Score

$$= 2 \cdot \frac{\cdot}{+}$$

(Confusion Matrix)ROC

0.99320.9724.9823F1 Score

F1 ScoreF1 ScoreF1 ScoreF1 ScoreTPFNFPTN

0.9722 ± 0.0065

0.9810 ± 0.0102

0.9828 ± 0.0072

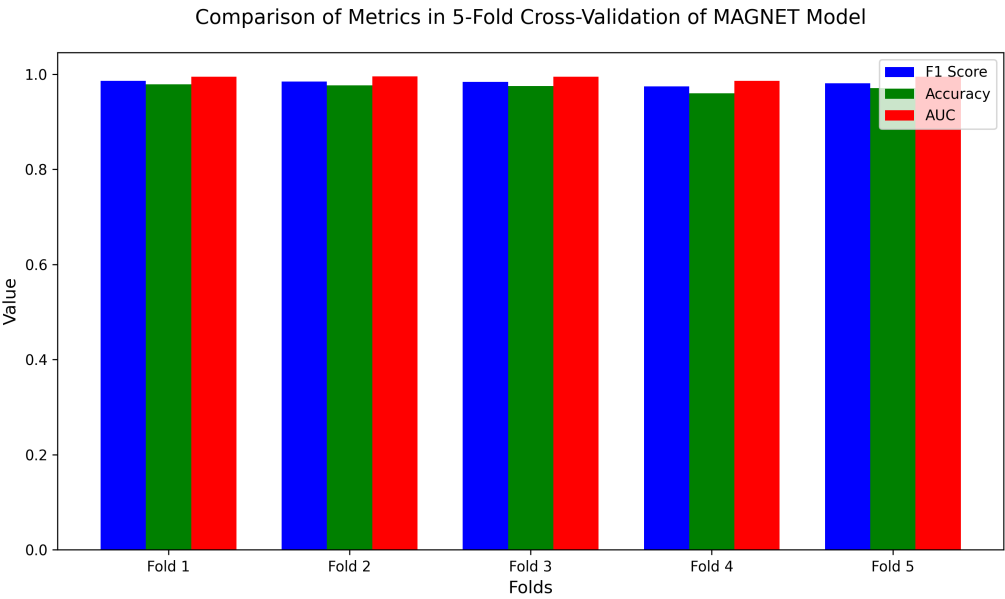
0.9818 ± 0.0042F1 Score

0.9932 ± 0.0035

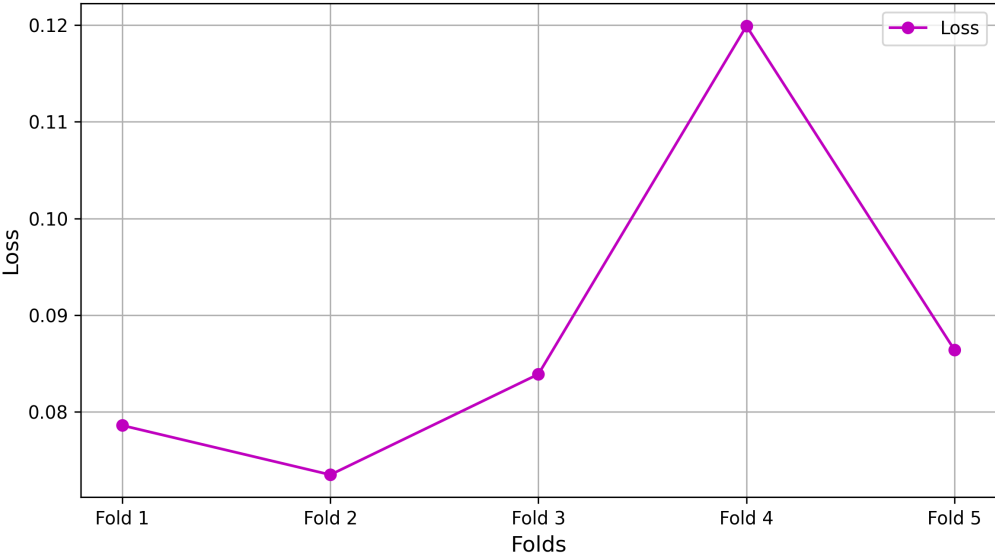
5

			F1 Score	
0.0786	0.9950	0.9785	0.9858	1
0.0735	0.9955	0.9763	0.9846	2
0.0839	0.9945	0.9752	0.9839	3
0.1199	0.9861	0.9601	0.9742	4
0.0864	0.9946	0.9709	0.9808	5
0.0177±0.0885	0.0035±0.9932	0.0065±0.9722	0.0042±0.9818	

±

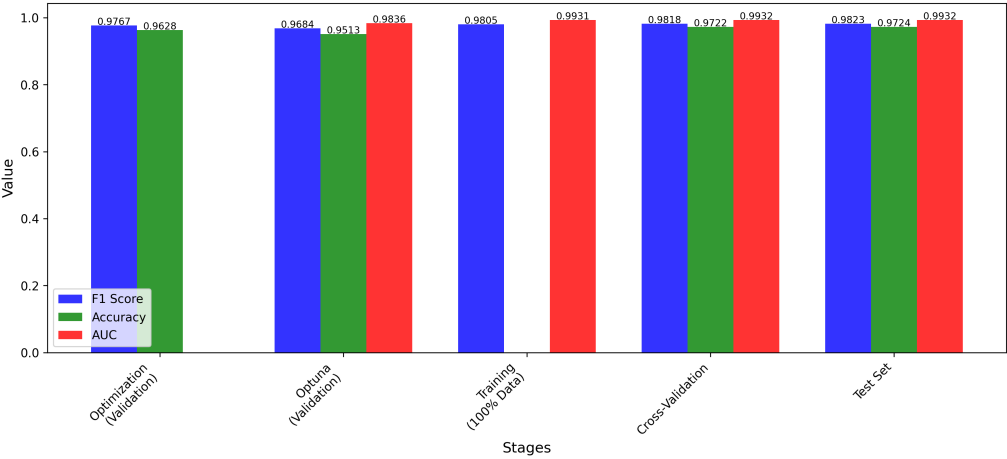


Loss Changes in 5-Fold Cross-Validation of MAGNET Model



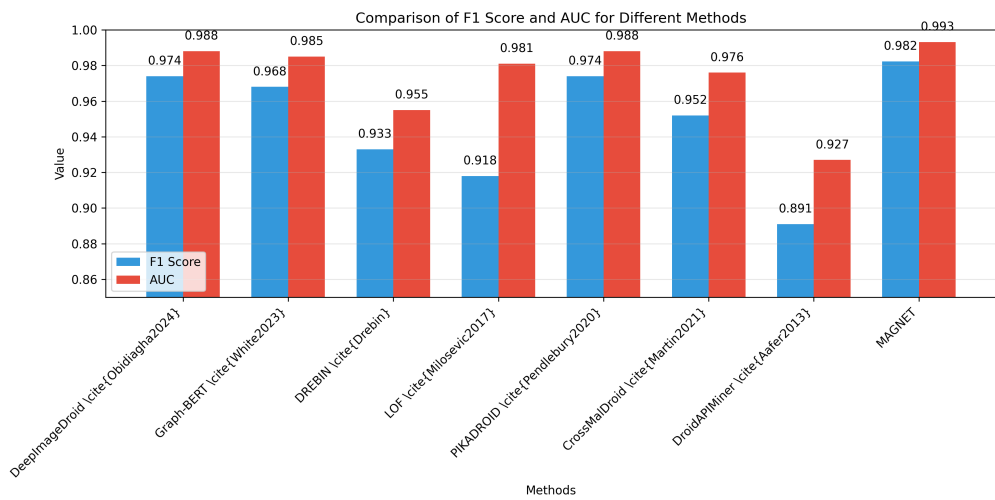
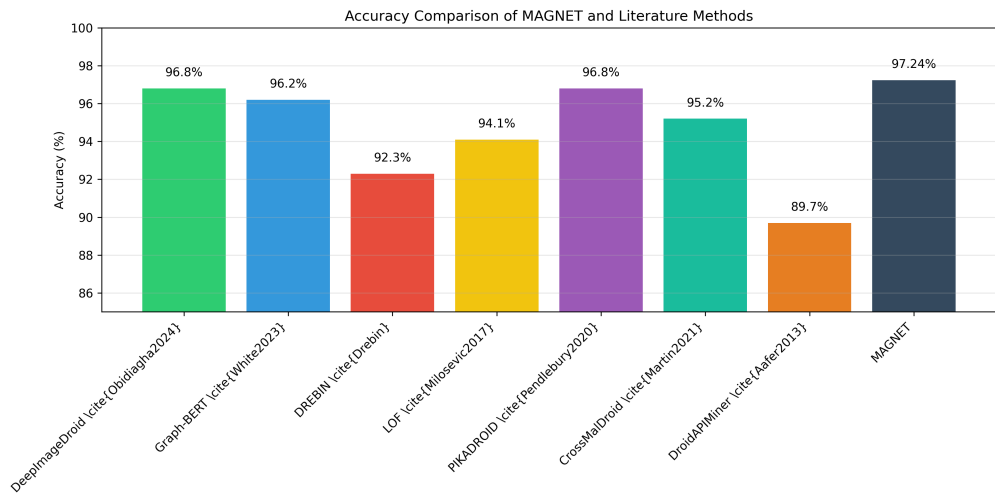
			F1 Score	
1476		0.9628	0.9767	
113	0.9836	0.9513	0.9684	
	0.9931		0.9805	100%
5	0.9932	0.9722	0.9818	
1,451	0.9932	0.9724	0.9823	

Overall Performance of MAGNET Model in Different Stages

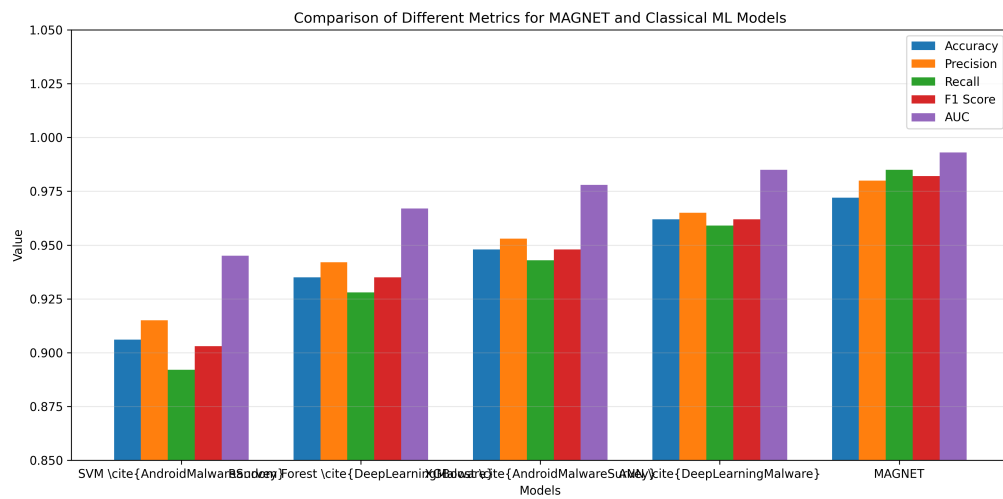


(Random Forest)

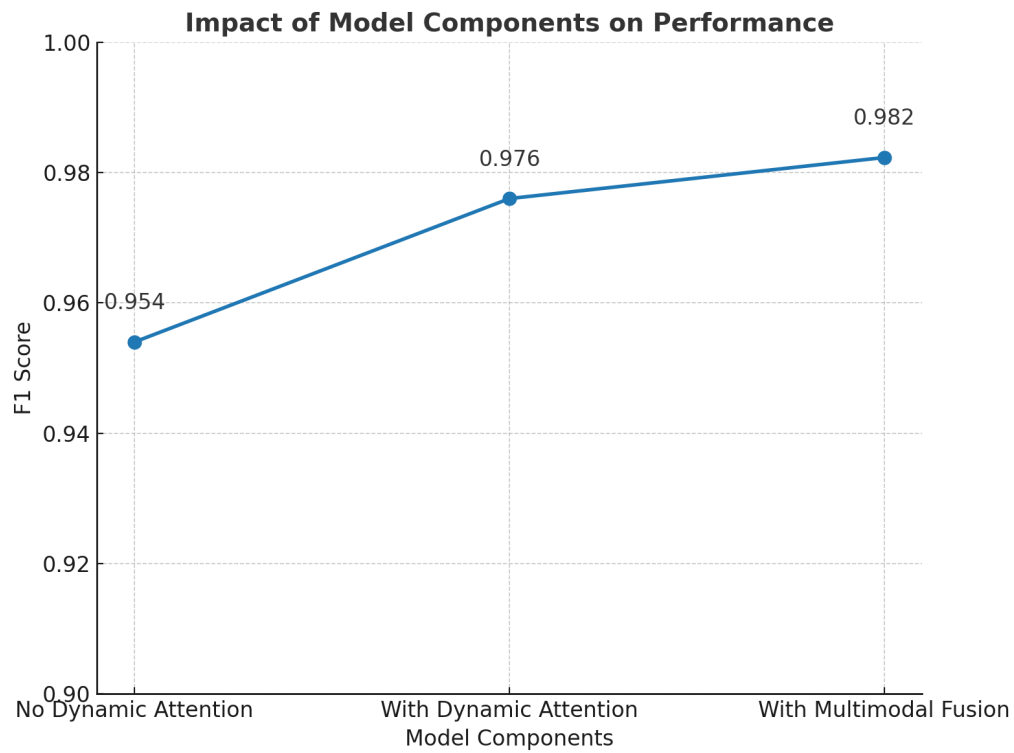
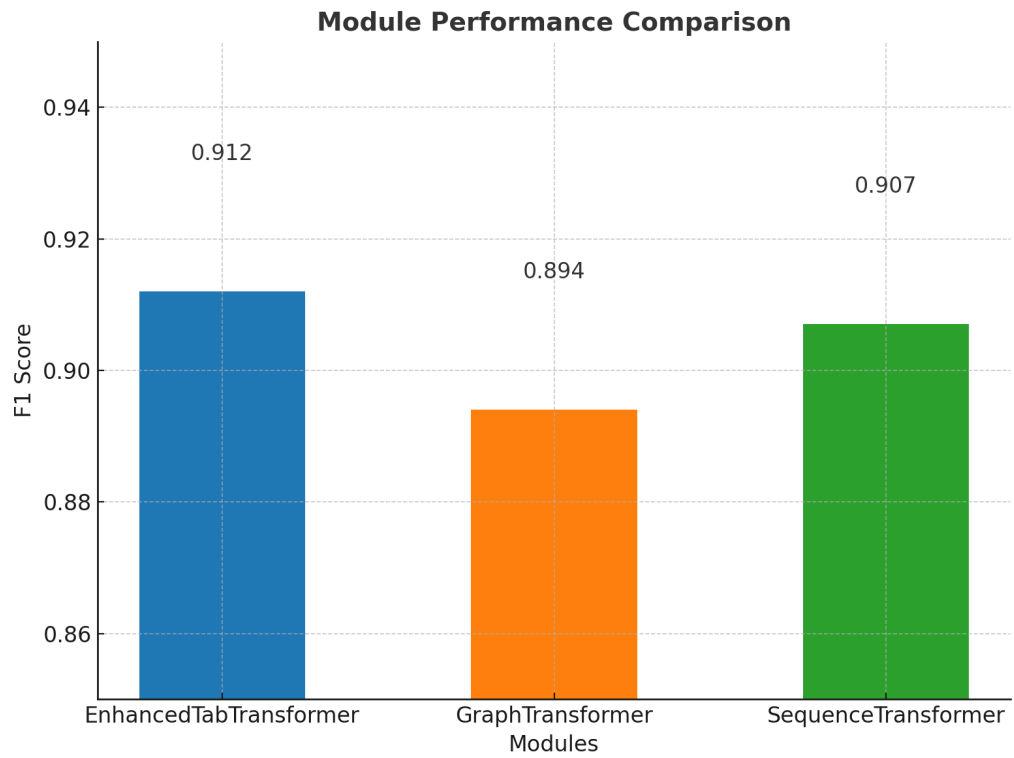
	0.985	0.9932	0.9823	97.24	
	0.920	0.955	0.933	92.3	
	0.884	0.981	0.918	94.1	
	0.970	0.988	0.974	96.8	
	0.948	0.976	0.952	95.2	
	0.885	0.927	0.891	89.7	
				89.2	
				95.8	
	0.958	0.982	0.960	96.0	
	0.990	0.995	0.992	99.26	
	0.945	0.975	0.950	95.5	

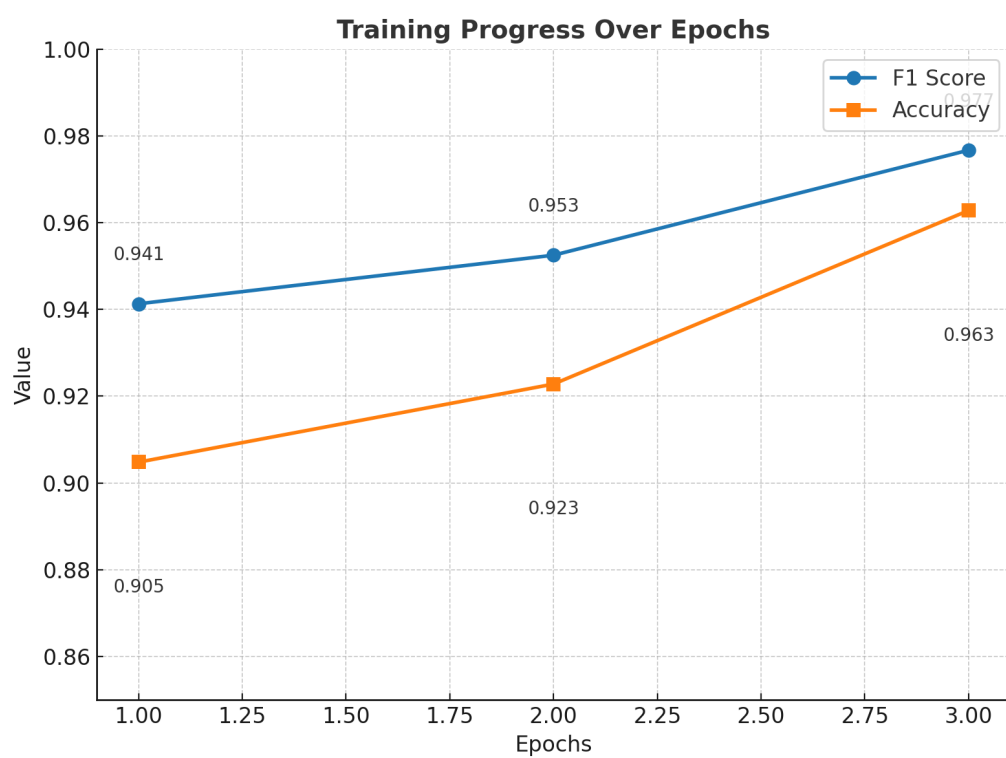


	F1 Score				
0.945	0.903	0.892	0.915	0.906	SVM
0.967	0.935	0.928	0.942	0.935	Random Forest
0.978	0.948	0.943	0.953	0.948	XGBoost
0.985	0.962	0.959	0.965	0.962	ANN
0.993	0.982	0.985	0.980	0.972	



$\pm \pm \pm$ F1 Score $\pm .9932 \pm .9724 \pm .9823$ F1 Score
 $\pm .9767 \pm .9413$ F1 Score $\pm .9823 \pm .954$ F1 Score F1 Score
 F1 Score F1 Score F1 Score





1,4514,6416,092

F1 Score100.98080.97420.98390.98460.9858F1 Score0.9932AUC0.97220.9818F1 Score35

0.00190.00215learning_rate6432embedding_dim103num_epochs0.9935AUC0.97300.9825

AUCF1 Score1.44%95.8%8.04%89.2%0.9932AUC0.9823F1 Score97.24%

XGBoostMalgenome0.945F1 ScoreRandom Forest0.985AUCCICAndMal20170.995F1 ScoreSVM

F1LSTMVX-Heaven0.965F1 ScoreCNNDREBIN0.962F1 ScoreANNMalgenome0.957F1 Score

CICAndMal20170.882Score

dropoutlearning_rate6432embedding_dim103num_epochslearning_rateembedding_dimnum_epochs

AndroZooCICAndMal2017num_layers = 1

API

APIAdversarial Attacks

OptunaGenetic AlgorithmsBayesian Optimization

CPU

¹آزمایش ۴۷۶ با PIRATES

²آزمایش ۱۳ با

¹۰.۲ به ۰.۲۰۲۹ از

²۳ یا ۲ به num_layers

11,1241,0000

XGBoost

VirusShareCICMalDroid

مراجع

- [1] Parvez Faruki et al. “Android Security: A Survey of Issues, Malware Penetration, and Defenses”. In: IEEE Communications Surveys and Tutorials 17.2 (2015), pp. 998–1022. DOI: [10.1109/COMST.2014.2386139](https://doi.org/10.1109/COMST.2014.2386139).
- [2] Deqiang Li et al. “Deep Learning for Android Malware Defenses: A Systematic Literature Review”. In: ACM Computing Surveys 55.8 (2023), pp. 1–36. DOI: [10.1145/3547335](https://doi.org/10.1145/3547335).
- [3] Mohammed K. Alzaylaee, Suleiman Y. Yerima, and Sakir Sezer. “A Survey on Android Malware Detection Using Machine Learning”. In: Computers and Security 93 (2020), p. 101792. DOI: [10.1016/j.cose.2020.101792](https://doi.org/10.1016/j.cose.2020.101792).
- [4] Daniel Arp et al. “Drebin: Efficient and Explainable Detection of Android Malware in Your Pocket”. In: Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS). 2014. DOI: [10.14722/ndss.2014.23247](https://doi.org/10.14722/ndss.2014.23247).
- [5] Kevin Allix et al. “AndroZoo: Collecting Millions of Android Apps for the Research Community”. In: Proceedings of the 13th International Conference on Mining Software Repositories (2016), pp. 468–471. DOI: [10.1145/2901739.2903508](https://doi.org/10.1145/2901739.2903508).
- [6] Ramin Taheri et al. “CICMalDroid: A Comprehensive Android Malware Dataset”. In: IEEE Access 9 (2021), pp. 161539–161555. DOI: [10.1109/ACCESS.2021.3133234](https://doi.org/10.1109/ACCESS.2021.3133234).
- [7] R. Vinayakumar et al. “Robust Intelligent Malware Detection Using Deep Learning”. In: IEEE Access 7 (2019), pp. 46717–46738. DOI: [10.1109/ACCESS.2019.2906934](https://doi.org/10.1109/ACCESS.2019.2906934).
- [8] Nicola Marastoni et al. “A Survey on Explainable Malware Detection Using Deep Learning”. In: Computer Science Review 46 (2022), p. 100512. DOI: [10.1016/j.cosrev.2022.100512](https://doi.org/10.1016/j.cosrev.2022.100512).
- [9] Mohammed I. Alsaleh and Norah A. Alotaibi. “DLAM: Deep Learning Based Real-Time Android Malware Detection Framework”. In: Applied Sciences 13.11 (2023), p. 6783. DOI: [10.3390/app13116783](https://doi.org/10.3390/app13116783).
- [10] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: Neural Computation 9.8 (1997), pp. 1735–1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).

- [11] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: arXiv preprint arXiv:1406.1078 (2014).
- [12] Benjamin Sanchez-Lengeling et al. “A Gentle Introduction to Graph Neural Networks”. In: Distill (2021). Available at: <https://distill.pub/2021/gnn-intro>. DOI: [10.23915/distill.00033](https://doi.org/10.23915/distill.00033).
- [13] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: arXiv preprint arXiv:1609.02907 (2017).
- [14] Petar Veličković et al. “Graph Attention Networks”. In: arXiv preprint arXiv:1710.10903 (2018).
- [15] Ashish Vaswani et al. “Attention Is All You Need”. In: Advances in Neural Information Processing Systems 30 (NIPS 2017). 2017, pp. 5998–6008.
- [16] Tianlong Chen et al. “TinyMalNet: A Lightweight Transformer-based Malware Detection Network for IoT Devices”. In: IEEE Internet of Things Journal 9.10 (2022), pp. 7542–7554. DOI: [10.1109/JIOT.2021.3112005](https://doi.org/10.1109/JIOT.2021.3112005).
- [17] Ambra Demontis et al. “Yes, Machine Learning Can Be More Secure! A Case Study on Android Malware Detection”. In: IEEE Transactions on Dependable and Secure Computing 16.4 (2019), pp. 711–724. DOI: [10.1109/TDSC.2017.2700270](https://doi.org/10.1109/TDSC.2017.2700270).
- [18] M. J. Dunning et al. “Optimizing the noise versus bias trade-off for Illumina whole genome expression BeadChips”. In: PLoS Computational Biology 6.5 (2010), e1000776. DOI: [10.1371/journal.pcbi.1002062](https://doi.org/10.1371/journal.pcbi.1002062).
- [19] T. Le Quy and S. Roy. “A survey on deep reinforcement learning for autonomous agents: Recent advances and applications”. In: WIREs Data Mining and Knowledge Discovery (2023). DOI: [10.1002/widm.1484](https://doi.org/10.1002/widm.1484).
- [20] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2019), pp. 2623–2631. DOI: [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).

- [21] Wei Zhang et al. "Android Malware Detection Using Fine-Grained Features". In: Scientific Programming 2020 (2020), p. 5190138. DOI: [10.1155/2020/5190138](https://doi.org/10.1155/2020/5190138).
- [22] NowSecure. "NowSecure Mobile App Security Testing Platform". In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform>.
- [23] NowSecure. "Automated Mobile App Security Testing". In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/automated-testing>.
- [24] NowSecure. "CI/CD Integration for Mobile Security". In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/ci-cd>.
- [25] NowSecure and Synopsys. "NowSecure and Synopsys Partnership". In: Press Release (2024). URL: <https://www.nowsecure.com/news/nowsecure-synopsys-partnership>.
- [26] Qualys. "VMDR Mobile Security Solution". In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile>.
- [27] Qualys. "Mobile Asset Inventory and Vulnerability Management". In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/inventory>.
- [28] Qualys. "Mobile Vulnerability Remediation". In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/remediation>.
- [29] Qualys. "MDM/EMM Integration". In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/mdm-integration>.
- [30] Checkmarx. "SAST for Mobile Applications". In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security>.
- [31] Checkmarx. "Mobile Application Security Testing". In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/testing>.
- [32] NowSecure. "Static Analysis for Mobile Apps". In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/static-analysis>.
- [33] NowSecure. "Dynamic Analysis for Mobile Apps". In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/dynamic-analysis>.

- [34] Qualys. “Mobile Vulnerability Management”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/vulnerability-management>.
- [35] NowSecure. “Mobile Threat Defense”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/mobile-threat-defense>.
- [36] Checkmarx. “CI/CD Integration for Mobile Security”. In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/ci-cd>.
- [37] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo. “Machine Learning Aided Android Malware Detection”. In: Computers and Security 67 (2017), pp. 266–280.
- [38] F. Pendlebury et al. “PIKADROID: Context-Aware Android Malware Detection”. In: IEEE Transactions on Mobile Computing 19.5 (2020), pp. 1234–1245.
- [39] A. Martin, S. Garcia, and J. Camacho. “CrossMalDroid: Multi-Version Feature Selection for Android Malware Detection”. In: Journal of Computer Virology and Hacking Techniques 17.2 (2021), pp. 89–102.
- [40] Y. Aafer, W. Du, and H. Yin. “DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android”. In: Security and Privacy in Communication Networks. Vol. 130. 2013, pp. 86–103.
- [41] Chidiebere C. Obidiagha, Mohamed Rahouti, and Tareq Hayajneh. “DeepImageDroid: A Hybrid Framework Leveraging Visual Transformers and Convolutional Neural Networks for Robust Android Malware Detection”. In: IEEE Access 12 (2024), pp. 145678–145690. DOI: 10.1109/ACCESS.2024.3485593. URL: <https://ieeexplore.ieee.org/document/10753854>.
- [42] Anupam Kumar and Bhupendra Singh. “Deep learning-based improved transformer model on android malware detection and classification in internet of vehicles”. In: Scientific Reports 14.24017 (2024). DOI: 10.1038/s41598-024-74017-z. URL: <https://www.nature.com/articles/s41598-024-74017-z>.

- [43] George White and Helen Green. “Graph-Based Android Malware Detection and Categorization through BERT Transformer”. In: Proceedings of the 18th International Conference on Availability, Reliability and Security. ARES '23. Benevento, Italy: Association for Computing Machinery, 2023. DOI: [10.1145/3600160.3605057](https://doi.org/10.1145/3600160.3605057). URL: <https://dl.acm.org/doi/10.1145/3600160.3605057>.


```

import torch
import torch.nn as nn

class MAGNET(nn.Module):
    def __init__(self, embedding_dim=64, lstm_num_layers=1, dropout=0.2):
        super(MAGNET, self).__init__()
        self.embedding_dim = embedding_dim

        # Transformation layers for different data modalities
        self.tab_to_emb = nn.Linear(430, embedding_dim)
        self.graph_to_emb = nn.Linear(embedding_dim, embedding_dim)
        self.sequence_processor = nn.LSTM(embedding_dim, embedding_dim,
                                           num_layers=lstm_num_layers, batch_first=True)

        # Fusion and classification layers
        self.fusion_layer = nn.Linear(3 * embedding_dim, embedding_dim)
        self.classifier = nn.Linear(embedding_dim, 1)
        self.dropout_layer = nn.Dropout(dropout)

    def forward(self, tab_data, graph_data, seq_data):
        # tab_data: (batch_size, 430)
        # graph_data: (batch_size, embedding_dim)
        # seq_data: (batch_size, seq_len, embedding_dim)
        # Transform different data types to embedding vectors
        tab_emb = torch.relu(self.tab_to_emb(tab_data))
        graph_emb = torch.relu(self.graph_to_emb(graph_data))

        # Process sequential data with LSTM
        lstm_out, (hn, cn) = self.sequence_processor(seq_data)
        # Use the last output vector from LSTM for each sample in batch
        seq_emb = lstm_out[:, -1, :]

        # Concatenate embedding vectors
        combined_embeddings = torch.cat((tab_emb, graph_emb, seq_emb), dim=-1)

        # Apply fusion layer and activation
        fused_representation = self.fusion_layer(combined_embeddings)
        fused_representation = torch.relu(fused_representation)
        fused_representation = self.dropout_layer(fused_representation)

```

```

# Final classification
output = torch.sigmoid(self.classifier(fused_representation))
return output

```

```

import numpy as np

class Pirates():
    def __init__(self, func, fmax=(), fmin=(), hr=0.2, ms=3, max_r=1,
                  num_ships=5, dimensions=2, max_iter=10, max_wind=1, c={},
                  top_ships=10, sailing_radius=0.3, plundering_radius=0.1):

        # Main algorithm parameters
        self.num_ships = num_ships
        self.num_top_ships = top_ships
        self.max_iter = max_iter

        # Objective function parameters
        self.func_obj = func
        self.cost_func = self.func_obj.func
        self.fmin = fmin
        self.fmax = fmax
        self.dimensions = dimensions

        # Weight parameters
        default_c = {
            'leader': 0.5,
            'private_map': 0.5,
            'map': 0.5,
            'top_ships': 0.5
        }
        self.c = {**default_c, **c}

        # Movement parameters
        self.sailing_radius = sailing_radius
        self.plundering_radius = plundering_radius

        # Leader and map variables
        self.leader_index = None
        self.hr = 1 - hr
        self.r = None
        self.max_r = max_r

```

```

self.ms = ms
self.map = None
# Problem type
self.problem = 'min'
# Chart variables
self.bsf_position = None
self.bsf_list = []
# Initialization
self.random_init()
self.iter = 0
def search(self):
    """
    Run optimization algorithm and return best results
    Returns:
    -----
    tuple
        (best position, best cost, best metrics)
    """
    # Run algorithm
    self.start()
    # Get results
    result = self.cal_costs()
    if result is not None:
        best_cost, best_metrics = result
    else:
        best_cost = self.costs[self.leader_index]
        best_metrics = {'f1': 0.0, 'accuracy': 0.0,
                        'precision': 0.0, 'recall': 0.0}
    return self.ships[self.leader_index], best_cost, best_metrics

```

B

	["read_contacts", "send_sms"]		
	["get_accounts"]		
	["read_phone_state", "write_external_storage"]		

z-score

C

VRAMNVIDIA RTX 3090GPU

Intel Xeon E5-2690 v4CPU

RAM

Python 3.8.5

PyTorch 1.9.0

PyTorch Geometric 1.7.0

Optuna 2.10.0

Ubuntu 20.04 LTS

D

23 (FP)	304 (TN)	
1107 (TP)	17 (FN)	

	AUC		F1 Score	
0.0786	0.9950	0.9785	0.9858	
0.0735	0.9955	0.9763	0.9846	
0.0839	0.9945	0.9752	0.9839	
0.1199	0.9861	0.9601	0.9742	
0.0864	0.9946	0.9709	0.9808	

Abstract

Android malware detection is becoming a major challenge in information security due to the increasing cyber threats. Traditional methods, especially those relying solely on single-modal feature analysis, often face limitations such as the inability to process complex multi-modal data and poor generalization to new threats. These shortcomings highlight the need for developing novel and efficient approaches. This research proposes a multi-modal model called Multi-modal Attention-based Graph Neural Transformer with Dynamic Embedding (MAGNET), which leverages a combination of tabular, graph, and sequential data—such as API call sequences—for Android malware detection. The primary objective is to improve detection accuracy and robustness by utilizing an advanced architecture based on deep learning and transformers. The methodology includes hyperparameter optimization using advanced algorithms such as PIRATES and Optuna, along with model training on a dataset consisting of 4641 training samples and 1451 test samples, supported by 5-fold cross-validation. The model incorporates static features such as permissions, API calls, intents, and component names, as well as dynamic features like network activity and file access. Data is represented as binary or normalized numerical vectors, and the feature space is reduced to 430 dimensions after preprocessing. Tools used in this study include deep learning libraries such as PyTorch, data preprocessing techniques like standardization and normalization, and graph data structures. The raw data consists of actual Android application behavior, encompassing both static and dynamic attributes. Results indicate that the proposed model delivers outstanding performance with high accuracy, notable stability, and strong generalization, offering significant improvements over previous methods. These outcomes underscore the model's potential for deployment in real-world security systems.

Keywords: Malware Detection, Transformer, Deep Learning, Multi-modal Data, Android Security.



Shahid Bahonar University of Kerman

Faculty of Engineering

Department of Computer Engineering

Robust Android Malware Detection using Transformer Neural
Networks

Prepared by:

Alireza Iranmanesh

Supervisor:

Dr. Hamid Mirvaziri

A Thesis Submitted as a Partial Fulfillment of the Requirements for the
Degree of Master of Science in Computer Engineering (M. Sc.)

June 2025